

V2 Search Benchmark Work Review

Algorithm 4, Failure Learning, DrugEye Benchmarking, and Complexity

medicine-search-clean

Engineering briefing

Abstract

This document summarizes the V2 search work completed today. It is written for an engineering review: plain language first, then implementation details, complexity analysis, benchmark results, category-level error rates, and concrete examples. The central point is that the work did not add manual lookup answers. The hardcoded alias path was removed, the failures were converted into general ranking and candidate-generation rules, and the benchmark was rerun on the manual set, a 6,000-row proportional sample, and the full 115,000-row V2 dataset.

1 Plain-Language Summary

Think of the search system as a pharmacist assistant. A user may type a medicine name with missing letters, wrong vowels, a keyboard slip, OCR mistakes, or extra strength words like 500mg. The assistant should find the right family, but it should also know when the input is dangerous and should not confidently pick one product.

Today we did four main things:

1. Cleaned the Algorithm 4 story after discovering that manual aliases would be cheating. The final code has no hardcoded manual-case mapping.
2. Turned the supplied mistake analysis into general rules: first-letter variants, stronger tie breakers, prefix-only penalties, and broader confusion-pair handling.
3. Reran Algorithm 4 on the manual 50 cases, the 6,000-row proportional sample, and the full V2 dataset.
4. Documented time and space complexity for all algorithms and benchmarked DrugEye on the same 6,000-row sample for external comparison.

The result is useful but not magic. Algorithm 4 is safer than Algorithm 2 and better than Algorithm 1/2 on full V2 aggregate scores. It slightly beats Algorithm 3 on Hit@1, but after the broader recommendation pass it trails Algorithm 3 on Hit@20 and behavior success. So Algorithm 3 remains the cleaner production baseline; Algorithm 4 is a serious optimization/research direction, not a final replacement.

2 What Changed Today

Work item	What changed	Evidence
Removed cheating path	The manual alias table and alias rescue path were removed from Algorithm 4.	rg found no alias mapping path in Algorithm 4.
Learned from supplied mistakes	Added general first-letter variants, same-position/length tie breakers, expanded M/N confusion, and prefix-only penalties.	evaluation/master_algorithms/algorithm_4_commercial_name_search.py
Manual rerun	Reran the 50 manually supplied failed cases after the recommendation pass.	Hit@1 58.00%, Hit@20 70.00%, unsafe 0.00%.
Sample rerun	Reran Algorithm 4 on the deterministic 6,000-row proportional sample.	Hit@1 81.12%, Hit@20 92.83%, unsafe 0.00%.
Full rerun	Reran Algorithm 4 on all 115,000 V2 cases.	Hit@1 81.38%, Hit@20 92.98%, behavior 93.07%.
DrugEye benchmark	Benchmarked the public DrugEye trade-name path on the same 6,000-row sample.	Hit@1 21.75%, Hit@20 30.53%, no-result 62.88%.
Documentation	Updated README, Algorithm 4 design, manual failure analysis, and complexity notes.	testingdatasetv2/docs/

Table 1: Work completed in today’s V2 review session.

3 Algorithms Compared

Label	Implementation	Role in the benchmark
Algorithm 1	Current app evaluator	Conservative existing app behavior, large candidate pool, good safety behavior.
Algorithm 2	External English fast algorithm	Strong typo retrieval, but unsafe confident top-1 behavior was high.
Algorithm 3	Master rank-fusion algorithm	Runs Algorithm 1 and Algorithm 2, fuses them, then applies safety gates.
Algorithm 4	Algorithm 2 plus family rescue/safety layer	Avoids Algorithm 1’s full query path, adds bounded family rescue and clarification gates.
DrugEye trade	Public DrugEye website trade-name search	External website baseline, sampled because it is a public serial website call.

Table 2: Human-facing algorithm names used in the V2 reports.

4 One Worked Example

Input: flacton

This is a useful example because it shows both progress and the remaining problem.

Field	Value
Input	<code>flacton</code>
Expected family	<code>FLECTOR</code>
Manual-set pattern	multi-substitution
Old Algorithm 1/2/3 rank	not found within top 20
Latest Algorithm 4 rank	2
Latest top 5	<code>FLUCUTN</code> , <code>FLECTOR</code> , <code>FRACTIN</code> , <code>FLATICON</code> , <code>SLAFTON</code>
Safety result	ambiguous, not confident

Table 3: The concrete example used for the step-by-step explanation.

Plainly: before today’s recommendation pass, all first three algorithms missed `FLECTOR` within the top 20 for `flacton`. After the general rescue changes, Algorithm 4 finds `FLECTOR` at rank 2. It still has a wrong top-1 neighbor, so the safety gate is important: the system must show candidates and avoid confident dispensing.

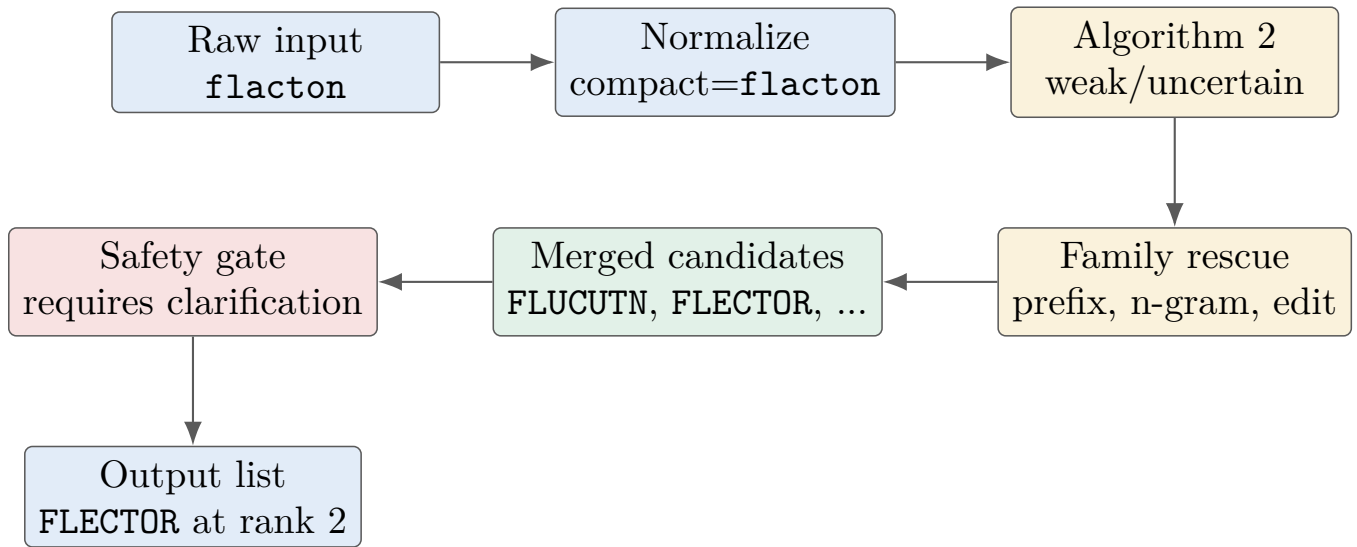


Figure 1: Worked example flow. The rescue layer improves retrieval, and the safety gate prevents an unsafe confident top-1.

5 Root-Cause Lessons Learned

Observed problem	Engineering lesson	Algorithm 4 action
Wrong answer is genuinely closer	Some expected values are family-equivalence problems, not search problems. Example: <code>levohista</code> is closer to <code>LEVOHISTAM</code> than <code>LEVOHISTAMIN</code> .	Document acceptable-alternative / family-grouping work instead of forcing a farther string.
Equal edit distance tie	Candidates can be equally close by edit distance, so the tiebreaker needs more evidence.	Add same-position score, length coverage, first-character preference, and full-word fuzzy evidence.
Correct family missing from candidate pool	Candidate generation was too narrow when the first letter was wrong.	Try plausible first-letter variants and broaden the family rescue prefilter.
Prefix-only false positive	A prefix or token fragment can look good while the full compact string is poor.	Penalize weak partial-prefix matches when edit evidence is low.
Fuzzy recovery can be dangerous	Finding a candidate is not the same as being safe to answer confidently.	Keep clarification required for fuzzy/risky recovered candidates.

Table 4: How the supplied mistakes were converted into general rules.

6 Benchmark Results

Full V2 Results

Table 5: Full V2 benchmark across every algorithm, with runtime and space columns added.

Algorithm	Cases	Recorded runtime	ms/case	Runtime note	Hit@1	Hit@20	Behavior	Unsafe	No result	Avg pool	Space meaning	Why this pool size happens
Algorithm 1	115,000	A1+A2 joint: 211.17s	1.84	Not isolated from Algorithm 2 in the recorded evaluator run.	75.25%	88.40%	88.66%	0.02%	2.82%	188.85	Largest temporary candidate set.	The current app evaluator keeps many product-row candidates before safety/ranking.
Algorithm 2	115,000	A1+A2 joint: 211.17s	1.84	Not isolated from Algorithm 1 in the recorded evaluator run.	79.29%	91.83%	91.28%	6.40%	1.09%	17.86	Smallest temporary candidate set.	The external fast algorithm is more aggressively indexed and usually returns a compact top candidate list.
Algorithm 3	115,000	233.84s	2.03	Standalone master run.	81.04%	93.09%	93.32%	0.00%	0.22%	37.69	Merged child candidate set.	It runs Algorithm 1 and Algorithm 2, then merges duplicate commercial families and applies safety gates.
Algorithm 4	115,000	335.60s	2.92	Standalone Algorithm 4 run after recommendation pass.	81.38%	92.98%	93.07%	0.00%	0.92%	22.39	Bounded rescue candidate set.	It keeps Algorithm 2 results, optionally adds context-clean results, then rescue-prefilters at most 40 families before returning top candidates.

Manual and Website Checks

Benchmark	Rows	Hit@1	Hit@20	Behavior	Unsafe	No result
Algorithm 4 manual failures	50	58.00%	70.00%	70.00%	0.00%	0.00%
Algorithm 4 proportional sample	6,000	81.12%	92.83%	92.90%	0.00%	0.83%
DrugEye trade sample	6,000	21.75%	30.53%	30.53%	0.00%	62.88%

Table 6: Additional checks run today. DrugEye is a public website baseline on the same 6,000-row sample, not a full local-catalog algorithm.

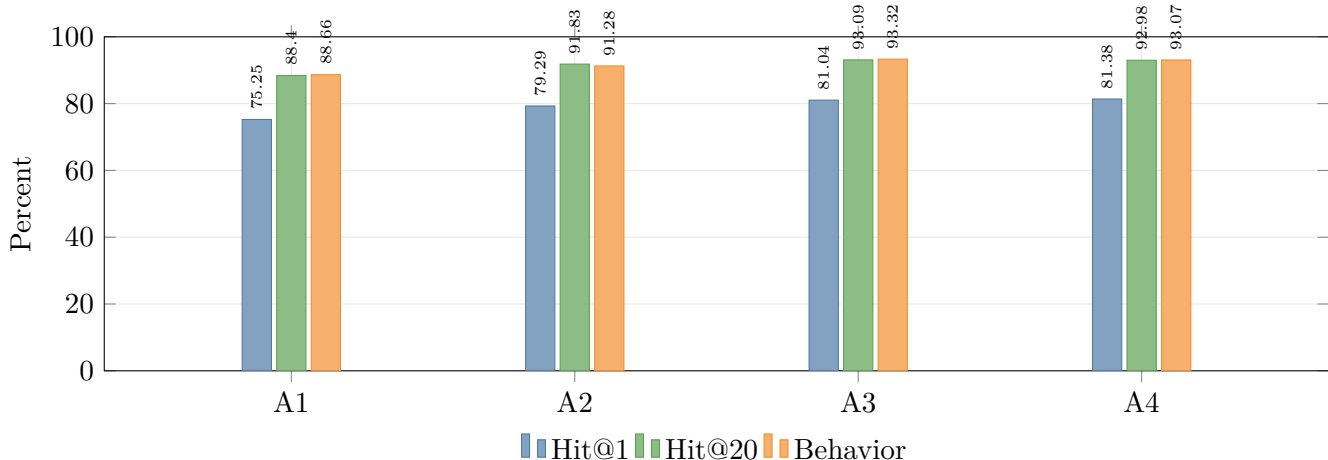


Figure 2: Full V2 quality comparison. The key tradeoff is Algorithm 4’s higher Hit@1 versus Algorithm 3’s slightly stronger Hit@20 and behavior success.

7 Time And Space Complexity

Use these symbols:

Symbol	Meaning
N	Product rows in the catalog.
F	Unique commercial families after grouping, $F \leq N$.
L	Average compact commercial-name length.
Q	Compact query length.
C_1	Algorithm 1 candidate count for one query.
C_2	Algorithm 2 candidate count for one query.
C_r	Algorithm 4 rescue family candidates after bucket collection and prefiltering.
M	Algorithm 3 merged child candidates, usually bounded by child top-k results.
R	Algorithm 4 merged candidate count.

Table 7: Complexity symbols.

Table 8: Detailed time, space, and quality analysis by algorithm.

Algorithm	Observed full-run time	Quality scores	Safety and failures	Observed query space	Preprocessed space	Per-query work and memory	Why the runtime/space looks like this
Algorithm 1	A1+A2 joint evaluator: 211.17s for 115,000 rows, 1.84 ms/case. A1 was not isolated in that run.	Hit@1 75.25%; Hit@20 88.40%; behavior 88.66%.	Behavior errors 13,045; no-result 2.82%; unsafe confident top-1 0.02%.	Average candidate pool 188.85, the largest observed pool.	$\mathcal{O}(NL^2)$ worst-case index space from product rows, normalized keys, prefix/delete-key style lookup data, and safety metadata.	$\mathcal{O}(C_1QL + C_1 \log C_1)$; keeps many product-row candidates while scoring and sorting. Temporary memory is approximately $\mathcal{O}(C_1)$.	Safer than Algorithm 2, but broad product-row recall means it carries far more candidates per query.
Algorithm 2	A1+A2 joint evaluator: 211.17s for 115,000 rows, 1.84 ms/case. A2 was not isolated in that run.	Hit@1 79.29%; Hit@20 91.83%; behavior 91.28%.	Behavior errors 10,030; no-result 1.09%; unsafe confident top-1 6.40%.	Average candidate pool 17.86, the smallest observed pool.	$\mathcal{O}(NL^2)$ worst-case index space plus typo features: n-grams, phonetic keys, skeleton keys, token statistics, and edit-distance support data.	$\mathcal{O}(Q^2 + C_2QL + C_2 \log C_2)$; memory approximately $\mathcal{O}(C_2)$.	Fast and compact because it aggressively narrows candidates, but it is willing to be confidently wrong more often.
Algorithm 3	Standalone master run: 233.84s for 115,000 rows, 2.03 ms/case.	Hit@1 81.04%; Hit@20 93.09%; behavior 93.32%.	Behavior errors 7,681; no-result 0.22%; unsafe confident top-1 0.00%.	Average candidate pool 37.69 after merging child candidates.	Algorithm 1 index space plus Algorithm 2 index space. It keeps both search stacks available.	Algorithm 1 query + Algorithm 2 query + $\mathcal{O}(M \log M)$ rank fusion; temporary memory is child query memory plus $\mathcal{O}(M)$.	More memory than Algorithm 2 because it merges two ranked lists, but stronger safety and behavior because Algorithm 1 can veto risky confidence.
Algorithm 4	Standalone Algorithm 4 run: 335.60s for 115,000 rows, 2.92 ms/case.	Hit@1 81.38%; Hit@20 92.98%; behavior 93.07%.	Behavior errors 7,965; no-result 0.92%; unsafe confident top-1 0.00%.	Average candidate pool 22.39. Smaller than Algorithm 3, larger than Algorithm 2.	Algorithm 2 index space plus $\mathcal{O}(FL^2)$ family rescue index over unique commercial families, not every product row.	Algorithm 2 query + optional context-clean query + gated rescue. Rescue prefilter keeps at most 40 families for expensive scoring; output is top 20. Temporary memory is about $\mathcal{O}(C_2 + C_r + R)$.	It keeps fewer final candidates than Algorithm 3, but it does heavier rescue scoring per uncertain query, so the current Python run is slower.

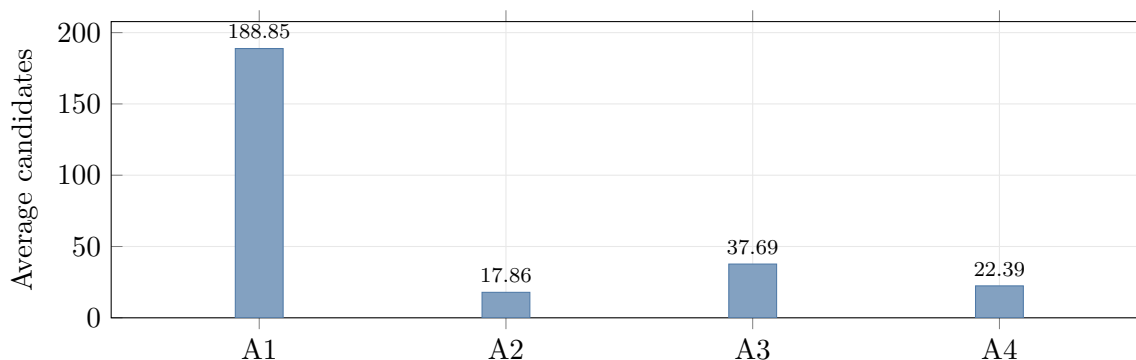


Figure 3: Average candidate pool on full V2. Algorithm 4 keeps a smaller pool than Algorithm 3, but its broader rescue logic made total runtime slower in the current Python benchmark harness.

8 Representative Test Cases

Input	Expected output	What it exercises
flacton	FLECTOR	Manual failure recovered by Algorithm 4 at rank 2 after general rescue changes.
Auticax	anticox	Compound typo with edit-distance tie pressure; still fails and needs stronger general multi-error handling.
levohista	levohistamin	Shows acceptable-alternative risk because LEVOHISTAM is genuinely closer than expected.
abc	__AMBIGUOUS__ or colliding family	Short prefix ambiguity; the safe behavior is clarification, not confidence.
112	__NO_MATCH__	Negative input that should not become a confident medicine match.
aaperenewalsunscreenspf 60ml	AAPERENEWALSUNSCREENSPF	Brand plus strength/form context; tests context cleanup.

Table 9: Examples that explain the important behavior, not just the aggregate scores.

9 Category Error Table

The table below compares all four algorithms on every V2 category. Hit@20 is the retrieval score. Error count and error rate are based on behavior success, so they include both retrieval misses and unsafe/wrong response-mode behavior.

Table 10: All-algorithm category-level scores, error counts, and error rates.

#	Scope	Category	Cases Example	A1 H@20	A2 H@20	A3 H@20	A4 H@20	A1 Err	A2 Err	A3 Err	A4 Err	A1 Err%	A2 Err%	A3 Err%	A4 Err%
1	inside	single_letter_visual_confusion	8,000 edwinerve → ADWINERVE(visual_A_to_E_pos_0)	96.79%	99.71%	99.46%	99.76%	257	23	43	19	3.21%	0.29%	0.54%	0.24%
2	inside	single_letter_phonetic_confusion	6,000 pabycare → BABYCARE(phonetic_B_to_P_pos_0)	99.58%	99.90%	99.90%	99.90%	25	6	6	6	0.42%	0.10%	0.10%	0.10%
3	inside	multi_char_phonetic_confusion	5,000 sharcodoll → CHARCODOLL(multi_phonetic_ch_to_sh_pos_0)	97.70%	99.72%	99.66%	99.90%	115	14	17	5	2.30%	0.28%	0.34%	0.10%
4	inside	ligature_confusion	4,000 dapom → ALAPOM(ligature_al_to_d_pos_0)	94.42%	97.32%	98.02%	99.33%	223	107	79	27	5.57%	2.68%	1.98%	0.68%
5	inside	single_char_deletion_position_weighted	5,000 3candyadults → D3CANDYADULTS(deletion_first_char_pos_0)	95.72%	99.88%	99.76%	99.92%	214	6	12	4	4.28%	0.12%	0.24%	0.08%
6	inside	single_char_insertion	3,000 asafetriumkids → SAFETRIUMKIDS(insertion_adjacent_key_letter_at_pos_0)	98.87%	99.97%	99.73%	99.97%	34	1	8	1	1.13%	0.03%	0.27%	0.03%
7	inside	transposition_position_weighted	3,000 abrmmon → BARMON(transposition_early_pos_0)	98.43%	99.83%	99.73%	99.83%	47	5	8	5	1.57%	0.17%	0.27%	0.17%
8	inside	truncation_doctor_abbreviation	6,000 abc → ABCIMMUNE(truncation_prefix_len_3)	80.83%	83.67%	85.92%	64.15%	1,150	980	845	2,151	19.17%	16.33%	14.08%	35.85%
9	inside	two_error_combinations	8,000 abgentp → APIGENTP(deletion_plus_phonetic_variant_0)	84.09%	93.77%	94.05%	98.16%	1,273	498	476	147	15.91%	6.23%	5.95%	1.84%
10	inside	three_error_combinations	5,000 acalagnhair → SCALOGENHAIR(keyboard_vowel_delete_variant_0)	54.08%	75.42%	77.36%	90.64%	2,296	1,229	1,132	468	45.92%	24.58%	22.64%	9.36%
11	inside	speed_typing_errors	3,000 aabilifymaintena → ABILIFYMAINTENA(double_first_letter)	89.97%	99.73%	99.67%	99.77%	301	8	10	7	10.03%	0.27%	0.33%	0.23%
12	inside	wrong_vowels_in_consonant_frame	5,000 aasyiron → EASYIRON(wrong_1_vowels_variant_0)	98.36%	98.78%	98.76%	99.22%	82	61	62	39	1.64%	1.22%	1.24%	0.78%
13	inside	punctuation_whitespace_copy_paste_artifacts	1,000 2hct → 2HCT(copy_paste_artifact_0)	100.00%	100.00%	100.00%	100.00%	0	0	0	0	0.00%	0.00%	0.00%	0.00%
14	inside	case_sensitivity	500 2hct → 2HCT(case_alternating_case)	100.00%	100.00%	100.00%	100.00%	0	0	0	0	0.00%	0.00%	0.00%	0.00%
15	inside	number_word_confusion	500 ALICOVITAMINDTHREE → ALICOVITAMIND3(number_word_variant_0)	97.20%	98.20%	98.40%	98.40%	14	9	8	8	2.80%	1.80%	1.60%	1.60%
16	semi_outside	embedded_form_strength_parsing	4,000 aaperenewalsunscreenspf60ml → AAPERENEWALSUNSCREENSPF60ML(brand_strength)	96.97%	87.67%	97.32%	99.40%	121	493	107	24	3.02%	12.32%	2.68%	0.60%
17	safety	dangerous_edl_pairs	5,000 aba → ALBA(dangerous_edl_deletion_variant_0)	83.86%	96.68%	94.48%	97.52%	807	166	276	124	16.14%	3.32%	5.52%	2.48%
18	safety	substring_traps	3,000 abimol → ABIMOLEXTRA(substring_long_truncated_to_short)	77.97%	79.40%	82.13%	78.93%	661	618	536	632	22.03%	20.60%	17.87%	21.07%
19	safety	negative_no_match_expected	3,000 112 → __NO_MATCH__(negative_catalog_metadata)	99.97%	99.83%	100.00%	100.00%	1	5	0	0	0.03%	0.17%	0.00%	0.00%
20	safety	contradictory_form_route	2,000 acetylcysteinevitaeardrops → ACETYLCYSTEINEVITA(contradictory_form_ear_drops)	99.95%	99.95%	99.95%	99.90%	1	1	1	2	0.05%	0.05%	0.05%	0.10%
21	safety	cancelled_na_drug_lookup	1,000 acnesol → ACNESOL(cancelled_exact)	99.40%	99.80%	99.70%	99.70%	6	2	3	3	0.60%	0.20%	0.30%	0.30%
22	safety	score_gap_ambiguity_detection	4,000 act → __AMBIGUOUS__(score_gap_edl_collision_proxy)	91.27%	86.05%	91.90%	84.47%	63	1,000	63	546	1.58%	25.00%	1.58%	13.65%
23	inside	ocr_letter_digit_confusion	4,000 99q999999rrr → 999999999RRR(ocr_9_to_Q_pos_2)	98.98%	99.85%	99.92%	99.85%	41	6	3	6	1.02%	0.15%	0.08%	0.15%
24	inside	ocr_plus_other_error_combine	3,000 Octomtol → OCTOMOTOL(ocr_plus_deletion_pos_0)	92.07%	94.27%	95.97%	98.60%	238	172	121	42	7.93%	5.73%	4.03%	1.40%
25	inside	double_letter_reduction_expansion	3,000 bbactroban → BACTROBAN(double_expansion_B_to_BB_pos_0)	99.87%	99.93%	99.90%	99.93%	4	2	3	2	0.13%	0.07%	0.10%	0.07%
26	inside	keyboard_adjacent_sampled	2,000 qbcimmune → ABCIMMUNE(keyboard_adjacent_A_to_Q_sampled_pos_0)	91.00%	98.80%	97.50%	98.80%	180	24	50	24	9.00%	1.20%	2.50%	1.20%
27	inside	four_plus_error_combinations	4,000 abksunbick → APICSUNBLOCK(ligature_two_phonetic_delete_variant_0)	50.82%	69.42%	72.78%	85.78%	1,967	1,223	1,089	569	49.18%	30.57%	27.22%	14.22%
28	inside	consonant_frame_wrong_vowels_heavy	4,000 2mwhytisbyagimyren → 2MWHITESBEEGUMARIN(all_vowels_wrong)	97.08%	97.95%	98.00%	98.83%	117	82	80	47	2.92%	2.05%	2.00%	1.18%
29	inside	multi_word_name_fragmentation	2,000 cald → CALNESIAD(abbreviated_tokens)	62.20%	70.80%	68.80%	69.75%	756	584	624	605	37.80%	29.20%	31.20%	30.25%
30	inside	autocorrect_artifacts	2,000 foster → FASTER(autocorrect_vowel_correction)	1.85%	3.60%	3.45%	3.50%	1,963	1,928	1,931	1,930	98.15%	96.40%	96.55%	96.50%
31	smoke	exact_match_baseline	2,000 2HCT → 2HCT(exact_base_group)	100.00%	100.00%	100.00%	100.00%	0	0	0	0	0.00%	0.00%	0.00%	0.00%
32	smoke	exact_match_with_strength	2,000 ABCMOCOTIVE7OML → ABCMOCOTIVE(exact_brand_with_real_strength)	100.00%	95.85%	100.00%	98.55%	0	83	0	29	0.00%	4.15%	0.00%	1.45%
33	smoke	keyboard_shift_whole_word	500 aahaxis → SAJACOD(keyboard_shift_left)	82.40%	0.40%	82.40%	1.40%	88	498	88	493	17.60%	99.60%	17.60%	98.60%
34	smoke	prefix_ambiguity_awareness	1,500 ace → __AMBIGUOUS__(ambiguous_prefix_len_3)	99.40%	99.73%	99.73%	97.67%	0	196	0	0	0.00%	13.07%	0.00%	0.00%

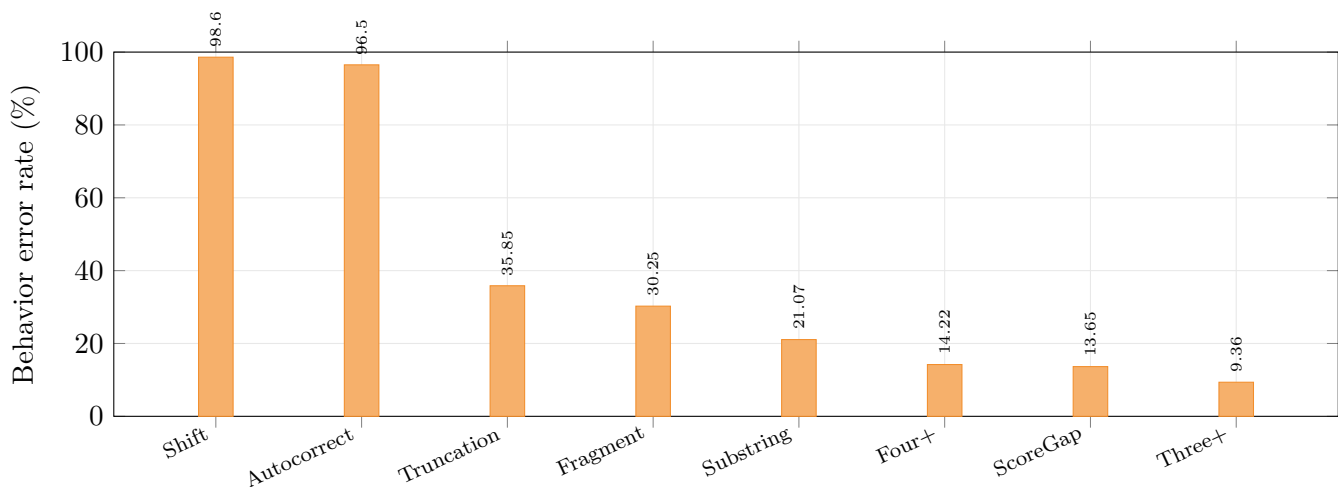


Figure 4: Highest Algorithm 4 behavior-error rates by category. The biggest remaining failures are whole-keyboard shifts, autocorrect artifacts, short truncations, and fragmented multi-word names.

10 Engineering Takeaways

- The most important correction was ethical and methodological: do not memorize the manual failures. The benchmark is only useful if the code learns general patterns.
- Algorithm 4 now learns from failure categories, not individual answers. The added rules are first-letter variants, tie-breaking evidence, partial-prefix penalties, and broader confusion groups.
- Algorithm 4 improves full Hit@1 over Algorithm 3, but Algorithm 3 still wins on Hit@20 and behavior success. That makes Algorithm 3 the safer current baseline.
- DrugEye performs much worse on the V2 sample, mostly because many noisy generated inputs produce no website result.
- The hardest remaining categories are not ordinary typos. They are whole-keyboard shifts, autocorrect artifacts, short truncations, fragmented multi-word names, substring traps, and score-gap ambiguity.
- Next work should focus on family-equivalence labels, acceptable alternatives, better multi-token compact matching, and a separate whole-keyboard-shift decoder.